CS23532 – COMPUTER NETWORKS
EX 6: Error Correction at Data Link Layer (Hamming Code)

AIM :

To write a program to implement error detection and correction using the Hamming code concept, and to make a test run with an input data stream to verify the error correction feature.

PROCEDURE :

Error Correction at Data Link Layer

Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Sender program

1. The input to the sender is a text of any length. The program converts the text to binary.
2. Apply the Hamming code concept on the binary data and add redundant bits to it.
3. Save this output in a file called channel.

Receiver program

1. The receiver program reads the input from the channel file.
2. Apply Hamming code on the binary data to check for errors.
3. If there is an error, display the position of the error.
4. Else, remove the redundant bits, convert the binary data to ASCII and display the output.

PROGRAM:

sender.py

```
# Hamming code - SENDER (even parity, Hamming(12,8) per character)
PARITY_POS = [1, 2, 4, 8]

def text_to_binary(text):
    return [format(ord(ch), '08b') for ch in text]

def hamming_encode(data_bits):
    code = [0] * 13                     # index 1..12 used
    d = 0
    for pos in range(1, 13):
        if pos not in PARITY_POS:
            code[pos] = int(data_bits[d])
            d += 1
    for p in PARITY_POS:                # even parity over covered positions
        code[p] = sum(code[i] for i in range(1, 13) if i & p and i != p) % 2
    return ''.join(str(b) for b in code[1:])

text = input("Enter the text to send: ")
binary = text_to_binary(text)
print("Binary data      :", ' '.join(binary))
codewords = [hamming_encode(b) for b in binary]
print("Hamming codewords:", ' '.join(codewords))
with open("channel", "w") as f:
    f.write('\n'.join(codewords))
print("Codewords saved in file 'channel'")
```

receiver.py

```
# Hamming code - RECEIVER (even parity, Hamming(12,8) per character)
PARITY_POS = [1, 2, 4, 8]

with open("channel") as f:
    codewords = [line.strip() for line in f if line.strip()]

message = ""
for n, cw in enumerate(codewords, 1):
    bits = [0] + [int(b) for b in cw]    # index 1..12
    syndrome = 0
    for pos in range(1, 13):
        if bits[pos]:
            syndrome ^= pos             # XOR of positions holding a 1
    if syndrome != 0:
        print(f"Codeword {n}: {cw} -> ERROR at bit position {syndrome}")
        bits[syndrome] ^= 1             # correct the erroneous bit
        print(f"Codeword {n}: corrected -> {''.join(map(str, bits[1:]))}")
    else:
        print(f"Codeword {n}: {cw} -> no error")
    data = ''.join(str(bits[p]) for p in range(1, 13) if p not in PARITY_POS)
    message += chr(int(data, 2))

print("Received text    :", message)
```

OUTPUT:

Input

Text entered at the sender: Hello

Sender output

```
Enter the text to send: Hello
Binary data      : 01001000 01100101 01101100 01101100 01101111
Hamming codewords: 000110011000 100111000101 000011001100 000011001100
110111001111
Codewords saved in file 'channel'
```

Receiver output – no error in the channel

```
Codeword 1: 000110011000 -> no error
Codeword 2: 100111000101 -> no error
Codeword 3: 000011001100 -> no error
Codeword 4: 000011001100 -> no error
Codeword 5: 110111001111 -> no error
Received text    : Hello
```

Receiver output – error induced in the channel file

Bit 7 of the second codeword in the channel file was flipped manually (100111000101 changed to 100111100101).

```
Codeword 1: 000110011000 -> no error
Codeword 2: 100111100101 -> ERROR at bit position 7
Codeword 2: corrected -> 100111000101
Codeword 3: 000011001100 -> no error
Codeword 4: 000011001100 -> no error
Codeword 5: 110111001111 -> no error
Received text    : Hello
```

RESULT:

Thus, a program to implement error detection and correction using the Hamming code was written and executed. The receiver detected the position of the induced error and corrected it, and the original message was recovered successfully.